

REGIONALNI CENTAR ZA PREDINKUBACIJU U PAMETNOJ INDUSTRIJI



Sufinancira
Europska unija



Projekt je financiran putem ITU
mehanizma u okviru ITP-a


VARAZDIN

foi SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAZDIN



GenAI and How to Use It in Developing Your Own Solutions?

Tomislav Peharda



Introduction

- In recent months, new tools have emerged that significantly transformed software development
- The rise of coding agents capable of turning textual instructions into usable features or products
 - Our focus will be on Claude Code
- These tools enable rapid prototyping, execution of tasks, and work on items that would otherwise be (possibly) deprioritized (for example, visualizations)
- With proper usage, coding agents can significantly increase efficiency and improve output quality



Objectives and outcomes

- Objectives:
 - Understand how organizations use coding agents in practice
 - Learn the core concepts and mechanisms behind modern coding agents
 - Become familiar with Claude Code and its primary workflows
- Outcomes (after this session, participants will be able to):
 - Identify tasks suitable for delegation to coding agents
 - Use Claude Code to prototype simple applications and automate routine tasks
 - Evaluate when and how coding agents can improve individual and team productivity



Table of contents

- Introduction
- Objectives and outcomes
- Coding agents
- Agents and tools (in 2026)
- How organizations' non-technical staff use (coding) agents
- Scope of practical exercises
- Claude Code overview + mechanisms + exercises
- Voice navigation
- Application deployment
- What to explore next?
- Personal outlook
- Build session



Coding agents

- Primarily LLM-powered tools that generate code based on user instructions
- Access to the entire codebase, allowing the agent to explore code, make decisions, and implement changes
- Human-in-the-loop experience that requires user guidance during execution
- Vibe coding refers to the use of coding agents by providing high-level intent or "vibes" rather than precise instructions – often indicating limited coding knowledge or minimal code review



Coding agents: timeline

- LLM UIs (phase 1)
 - Example: ChatGPT
- IDEs (phase 2)
 - Example: Cursor
- **Agents (phase 3)**
 - Example: Claude Code



Agents and tools (in 2026)

- A chatbot refers to an LLM that generates text as output
- An agent refers to an LLM-powered chatbot that executes actions on the user's behalf
- Actions (or business logic) are typically implemented as tools containing business logic
- The agent is provided with instructions (a prompt) that define when and how specific tools should be used

Agents and tools (in 2026)

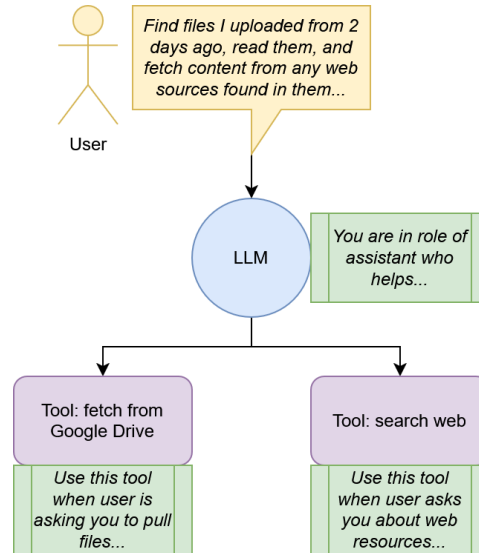


Figure 1. Agents and tools cohesion



How organizations' non-technical staff use (coding) agents?

- **Processes**
 - Boosting efficiency with repeated, low-risk tasks
 - Building enterprise knowledge
 - Building internal applications
 - Validating ideas
- **Activities**
 - Workshops and trainings
 - Hackathons
- **Goals**
 - Thoroughly validated ideas before actual implementation
 - Boost efficiency and quality in task execution
 - Internal applications tailored to their needs and expectations



Enterprise knowledge

- Company-wide knowledge accessible through a single, usually chat-based, experience
- Knowledge is built from a variety of data sources (Notion, email, Google Drive, etc.)
- Examples: Claude, ChatGPT (via connectors), Glean



Scope of practical exercises

- Tasks execution
 - For example, analyzing a dataset or creating a visualization
 - Enables execution of tasks based on natural language instructions that that previously required significantly more time (or would not have done in the first place)
- Prototyping ideas
 - Such as, building a simple web application
 - Allows non-technical audience to explore ideas that might grow into larger applications in the future

Scope of practical exercises: prototyping ideas



Figure 1. Idea development timeline



Claude Code overview

- Terminal-based coding agent from Anthropic
- First released in February 2025, with most adoption and popularity occurring in recent months
- Runs locally on your machine rather than in the browser, which allows access to your operating system and files
- The agent has a limited set of tools (such as bash, human-in-the-loop confirmation, and web search)
 - This demonstrates how even simple toolsets can drive significant impact



Installation

1. Download artifacts for conducting exercises (<https://tinyurl.com/foi-radionica>)
2. Navigate to Claude Code CLI docs (<https://code.claude.com/docs/en/quickstart>)
3. Open a terminal and setup api key (see *setup.txt* for instructions)
4. Follow the instructions to download and install Claude Code on your PC
5. *(There may be additional steps to complete in the terminal)*
6. Once installation is complete, type "claude" in the terminal (press Enter afterwards) to start
7. *(You will be asked to authenticate, in which case select Anthropic Console Account, and create a new account – do not add any credits)*
8. Enter your first instruction or query
9. *(Depending on a task you give to Claude Code, you might be asked to install additional libraries to execute tasks)*



Exercises

1. PDF modification: instruct agent to append new page with text "hello" to PDF at hand and store it
2. Dataset exploration: guide agent to analyze dataset, visualize it, and transform
3. Web application: provide instructions to build a simple web application



Pricing

- Token-based consumption which reflects on your subscription or API key credits
- If used extensively (every day), cost can easily grow over \$100 (even with a subscription)
- Offers workspace plans that come with other benefits



From prompts to specifications

- Paradigm shift from writing prompts to writing specifications
- Specification in detail describes key aspects for the implementation → increases quality (less hallucination, less ambiguity)
- Tooling (such as Spec Kit) interviews the user to build a specification, which is then passed into the agent for execution



Plan mode

- Thorough analysis and research before making an execution plan
- Generated plan (with alternatives) provided to the user for confirmation or adjustment
- Execution of the confirmed plan with higher degree of quality (iterative verification of plan milestones completion)



CLAUDE.md

- Memory implementation of agents is still in early phases and fragile, thus agents store their key knowledge in markdown files
- To have Claude Code follow user preferences (but also to have additional knowledge or context), putting valuable insights and details in *CLAUDE.md* file of the working directory will inject it into (every) LLM call
 - For instance, storing steps to connect to tricky VPN server



Skills

- Custom instructions defined by the user that Claude Code can apply when relevant
 - Similar to *CLAUDE.md*, but shared across sessions and directories
- For example, if you have a specific documentation style, you can define it as a skill and reuse it consistently



MCP

- MCP is a protocol that agents use to communicate with external APIs
- MCP server hosts tools that API providers expose (to conduct business logic)
- REST API → MCP



Voice navigation

- Transition from typing instructions to speaking them out
- Wispr Flow is tool that enables speech-to-text conversation within any application



Application deployment

- Some applications users build might only be useful to them, however, some might want to be used with others, which requires deployment to live environments
- New providers offer an easy way to deploy applications without significant overhead, even for non-technical users
 - Vercel
 - Railway



What to explore next?

- Anthropic Cowork
- Warp terminal
- OpenClaw
 - MoltBook
- v0 by Vercel
- Phone integration



Personal outlook

- Smaller companies getting traction over big ones due to higher degree of agility
- AI generating a lot of noise if misused (AI slop)
- Software engineering going through transition (responsibilities, tasks, mindset, etc.)
- Personal view on timeline
 - Next 2-3 years: building a lot of software
 - Next 3-5 years: building coordinated / cohesive systems (multi-agent systems)
 - After: piloting and maintaining systems



Thank you!



Build session

- Explore your ideas, and build a prototype